

# PromptMeGood Definitive Polish Plan

This document outlines the exact code-level fixes required to resolve the aesthetic, UX, and workflow issues currently affecting the live PromptMeGood site. These fixes are based on a deep audit of the live deployed scripts and the GitHub repository source code.

## 1. The Workflow Failures (API Mismatches)

The primary reason workflows are failing is a mismatch between the endpoints the frontend scripts are calling and the endpoints the backend API server actually exposes.

### The Problem:

The live frontend scripts ( `pmg-visual-studio.js` and `pmg-storyboard.js` ) are making `fetch` calls to endpoints that do not exist on the server, resulting in 404 Not Found errors.

- `pmg-visual-studio.js` calls `/api/vision-analyze` and `/api/video` .
- `pmg-storyboard.js` calls `/api/storyboard` .

However, the backend API server ( `api-server/src/routes/ai.ts` ) only exposes the following relevant endpoints:

- `/api/analyze` (not `/api/vision-analyze` )
- `/api/image`
- `/api/generate-prompt`
- `/api/refine-prompt`
- `/api/image-prompt`

There is no `/api/video` or `/api/storyboard` endpoint defined in the backend.

### The Fix (For Replit):

#### 1. Fix the Vision Analyze Endpoint:

In `public/scripts/pmg-visual-studio.js` , change the `fetch` call in `handleReverseImage` from `/api/vision-analyze` to `/api/analyze` . The backend already has a robust `/api/analyze` endpoint that handles image uploads and uses the vision model.

#### 2. Build the Missing Endpoints:

The backend `api-server/src/routes/ai.ts` must be updated to include the missing endpoints:

- **`/api/video`** : Implement the Sora video generation endpoint as previously discussed.
- **`/api/storyboard`** : Implement the endpoint that takes a concept and returns 5 sequential image prompts.

## 2. The Visual Tab Mobile Layout Bug

### The Problem:

On mobile, when the user taps the "Visual" tab in the bottom dock, the tab highlights, but the screen continues to show the Workstation content. The Visual Studio modal never mounts inline.

### The Cause:

In `pmg-chassis-v2.js`, the `wireMobileDock` function correctly sets the `data-pmgv2-mobile-tab` attribute on the `<html>` element. However, the CSS rules in `pmg-chassis-v2-full.css` that are supposed to show the `.pmgv2-tools` column (where the Visual Studio mounts inline) are being overridden or are insufficient.

Specifically, `html.pmg-chassis-v2[data-pmgv2-mobile-tab="suite"].pmgv2-tools { display: flex; }` is present, but the inline mount logic in `pmg-visual-studio.js` ( `inlineMount` ) relies on `matchMedia('(max-width: 900px)')` and a `MutationObserver` on the `data-pmgv2-mobile-tab` attribute.

The critical failure is that the `pmgv2-tools` column itself is hidden on mobile by default, and the inline mount logic appends the modal to it, but if the column isn't visible, the modal isn't visible.

### The Fix (For Replit):

In `public/styles/pmg-chassis-v2.css` (or `pmg-chassis-v2-full.css`), ensure the following rule is strictly enforced and not overridden by later media queries:

CSS

```
@media (max-width: 900px) {  
  html.pmg-chassis-v2[data-pmgv2-mobile-tab="suite"] .pmgv2-tools {  
    display: flex !important;  
    flex-direction: column;  
    width: 100%;  
  }  
  html.pmg-chassis-v2[data-pmgv2-mobile-tab="suite"] .pmgv2-main {  
    display: none !important;  
  }  
}
```

## 3. The Invisible Mobile Footer

### The Problem:

The footer links (Privacy, Terms, Contact) are completely invisible on mobile devices.

### The Cause:

In `pmg-chassis-v2-full.css`, around line 408, there is a rule that aggressively hides the contents of the right side of the status bar on mobile:

CSS

```
html.pmg-chassis-v2 .pmgv2-statusbar-r > span,  
html.pmg-chassis-v2 .pmgv2-statusbar-r > button { display: none !important; }
```

While this hides the spans and buttons, the `.pmgv2-foot-links` container is also affected by surrounding flex layout issues when the other elements are hidden.

### The Fix (For Replit):

Ensure the `.pmgv2-foot-links` container is explicitly set to display on mobile within the status bar.

CSS

```
@media (max-width: 900px) {  
  html.pmg-chassis-v2 .pmgv2-statusbar-r {  
    display: flex !important;  
    justify-content: center !important;  
    width: 100%;  
  }  
  html.pmg-chassis-v2 .pmgv2-foot-links {  
    display: flex !important;  
    justify-content: center;  
    gap: 12px;  
  }  
}
```

## 4. The Intrusive Mobile Composer

### The Problem:

The prompt composer takes up too much vertical space on mobile, even when the user is just scrolling or viewing results.

### The Cause:

The "collapsed pill" logic exists in `pmg-chassis-v2.js` (the `pmgv2-composer-tab` element), but the CSS transitions and height constraints in `pmg-chassis-v2-full.css` are not fully collapsing the main composer wrapper when the tab is not expanded.

### The Fix (For Replit):

Implement the strict "Expand on Demand" CSS provided in the previous handoff. The composer wrapper must have `height: 0` and `overflow: hidden` when not active on mobile.

#### CSS

```
@media (max-width: 768px) {  
  .pmgv2-composer-wrap:not(.is-open) {  
    height: 0 !important;  
    padding: 0 !important;  
    overflow: hidden !important;  
    border: none !important;  
  }  
}
```

## Summary for Replit

1. **API Routing:** Update frontend `fetch` calls to match backend routes (e.g., `/api/vision-analyze` -> `/api/analyze` ). Build the missing `/api/video` and `/api/storyboard` endpoints on the backend.
2. **Mobile Visual Tab:** Fix the CSS media queries to ensure `.pmgv2-tools` is `display: flex !important` when `data-pmgv2-mobile-tab="suite"` .
3. **Mobile Footer:** Ensure `.pmgv2-foot-links` is explicitly visible in the mobile status bar CSS.
4. **Mobile Composer:** Enforce strict `height: 0` on the composer wrapper when collapsed on mobile.